DEBUGGING

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 1: Narrow the Bug Before You Prompt

- **Before prompting, shrink the problem along three axes:**
- In history: use git bisect to find the commit where the test started failing.
- In input: reduce the failing input to a minimal case that still fails.
- In code: trace the backward slice on the wrong value; prompt with only the statements that can affect it. Paste the narrowed artifact, not the whole project.

Guideline 1: Reasoning

- Bounding the failure is the step that most often unblocks debugging; missing it is the biggest time sink
- Delta debugging (ddmin): halve input, rerun, keep failing half, recurse to a minimal case
- Program slicing: skilled programmers reason about a backward slice, not the full program
- Both techniques concentrate signal and strip noise, exactly what an LLM needs
- Without this step, models produce confident but wrong fixes by guessing at the symptom

Guideline 1: Key Practices - Prompting

- Bisect in history first if the bug is a regression. git bisect plus a one-line test finds the offending commit in logarithmic time.
- Minimize the input using ddmin: delete half, rerun, keep the failing half, recurse. Paste the minimal reproducer.
- Minimize the code: include only the backward slice on the wrong value. Delete unrelated imports, helpers, and branches

Guideline 1: Key Practices - Reviewing

- If the AI asks a clarifying question, treat it as a signal your reproducer was still too big or too vague. Shrink further instead of answering in prose.
- If the AI suggests changes to code you did not include in the prompt, reject that suggestion. It is guessing outside its context.
- Before accepting any fix, re-run your minimal reproducer and confirm the input now gives the expected output.

Guideline 2: Prompt With Observed Runtime State

- When the failure is about wrong values, do not hand the AI the source and ask why.
- First state the invariant you expect to hold. Set a conditional breakpoint on its negation.
- Run to the breakpoint, print the relevant variables, and paste the invariant plus the observed state plus the source into your prompt.
- The AI should diagnose a specific concrete execution, not simulate your program in its head.

Guideline 2: Reasoning

- Debugger-assisted LLMs produce more accurate and more explainable fixes
- Design-by-Contract provides the vocabulary: pre-conditions, post-conditions, and loop invariants name what should be true at a given point
- A prompt naming an invariant and showing its violation is a sharp, falsifiable question
- Concrete state is the main defense against hallucinated APIs and fabricated reasoning

Guideline 2: Key Practices - Prompting

- Write the invariant first in one sentence, before you open the debugger. Example: 'at line 42, lo <= hi should hold'
- Set a conditional breakpoint on the negation of the invariant. In pdb: b 42, not (lo <= hi). It only fires when the invariant actually breaks.
- At the breakpoint, print the variables and copy the debugger output verbatim into your prompt. Do not paraphrase.
- If a structure is large, include a summary plus the one or two elements that look wrong, not the whole object.

Guideline 2: Key Practices - Reviewing

- The AI's explanation should refer to the specific observed values you pasted. If it reasons from hypothetical values, push back: 'use the values I gave you.'
- Before applying a fix, set a second breakpoint after the change and verify by hand that the invariant now holds.
- Reject fixes that reference variables not in scope at your breakpoint. This is a common hallucination mode that the debugger catches immediately.

Guideline 3: Get Ranked Hypotheses, Then You Pick

- When a bug is not trivially reproducible, do not ask 'what's wrong, fix it.'
- Ask for a ranked list of hypotheses, each with an explicit reason tied to concrete evidence.
- You pick which hypothesis to test first. Write your reason for picking in your next prompt, and ask the AI to push back on that reason.
- Only after the pick and the pushback do you ask for the falsifying experiment.

Guideline 3: Reasoning

- Zeller's scientific debugging: the tool generates candidate theories, the human scientist picks which theory to test based on evidence and prior knowledge
- Handing the pick to the model recreates the automation-bias failure mode: developers accept the first plausible suggestion and stop investigating
- Writing a reason surfaces private context the AI cannot know: recent commits, already-tried paths, environment quirks

Guideline 3: Reasoning (continued)

- Writing a reason gives the AI something falsifiable to push back on, enabling real disagreement rather than sycophantic agreement
- The value of LLM-assisted debugging is in the explanations, not the verdicts
- Explanations are where the developer can catch the model being wrong
- Developers prefer a small number of step-by-step explanations over a long ranked list of suspicious lines

Guideline 3: Key Practices - Getting Hypotheses

- Require a ranked list of at least three hypotheses, each with an explicit reason tied to specific evidence in the code, the error, or the observed state.
- Ask the AI to state a prediction for each hypothesis that would hold if the hypothesis is true.
- Forbid the AI from proposing a patch at this stage.

Guideline 3: Key Practices - Choosing and Testing

- Read the AI's reasoning against what you privately know: recent commits, test history, environment, earlier ruled-out paths.
- Write your reason for your pick, citing something concrete: a line number, log entry, or commit. 'It seemed right' does not pass this bar.
- Ask the AI to push back on your reason before it proposes an experiment. Treat sycophantic agreement as a signal to re-prompt.
- Ask for the cheapest experiment that distinguishes your hypothesis from its closest alternative, not one that merely confirms your pick.
- Only accept a patch after the experiment confirms the hypothesis.

References

- [1] Layman et al. Debugging Revisited: Toward Understanding the Debugging Needs of Contemporary Software Developers. ESEM 2013. DOI: 10.1109/ESEM.2013.43
- [2] Bajpai et al. Let's Fix this Together: Conversational Debugging with GitHub Copilot. VL/HCC 2024. IEEE: 10714559
- [3] Kang et al. Explainable Automated Debugging via LLM-Driven Scientific Debugging. Empir. Softw. Eng. 2025. DOI: 10.1007/s10664-024-10594-x
- [4] Chen et al. Teaching Large Language Models to Self-Debug. arXiv:2304.05128. ICLR 2024.
- [5] Perry et al. Do Users Write More Insecure Code with AI Assistants? CCS 2023. DOI: 10.1145/3576915.3623157

References (continued)

- [6] Zeller. Why Programs Fail: A Guide to Systematic Debugging, 2nd ed. Morgan Kaufmann, 2009.
- [7] Yang et al. Patch Correctness Assessment: A Survey. ACM TOSEM 2024. DOI: 10.1145/3702972
- [8] Liu et al. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. arXiv:2404.00971
- [9] Kang, An, Yoo. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. FSE 2024. DOI: 10.1145/3660771
- [10] Zeller and Hildebrandt. Simplifying and Isolating Failure-Inducing Input. IEEE TSE 2002. DOI: 10.1109/32.988498

References (continued)

- [11] Weiser. Program Slicing. IEEE TSE 1984. DOI: 10.1109/TSE.1984.5010248
- [12] Zhong et al. Debug like a Human: LLM Debugger via Verifying Runtime Execution. ACL Findings 2024. DOI: 10.18653/v1/2024.findings-acl.49
- [13] Meyer. Applying Design by Contract. IEEE Computer 1992. DOI: 10.1109/2.161279
- [14] Parnin and Orso. Are Automated Debugging Techniques Actually Helping Programmers? ISSTA 2011. DOI: 10.1145/2001420.2001445
- [15] Tian et al. DebugBench: Evaluating Debugging Capability of LLMs. ACL Findings 2024. DOI: 10.18653/v1/2024.findings-acl.247